

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	08/4/2026	DATE SUBMITTED:	08/4/2026
NAME:	Jiro Luis A. Katindig	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 4 **Stacks**

1. PROCEDURE OUTPUT

Tasks & Intended Learning Outcomes (ILOs):

ILO A: Implement a stack using the C++ STL (std::stack).

Whole code for ILO A

```

1  #include <iostream>
2  #include <stack>
3
4  int main(){
5
6      std::stack<int> stack1;
7      std::cout<<"----Testing the stack STL----" <<std::endl;
8
9      std::cout<<"\n===== STACK 1 =====" <<std::endl;
10
11     //isEmpty
12     std::cout<<"Is the stack 1 empty? " << stack1.empty() << std::endl;
13
14     //push
15     stack1.push(10);
16     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
17     stack1.push(9);
18     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
19     stack1.push(8);
20     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
21     stack1.push(7);
22     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
23
24     stack1.pop();
25     std::cout << "Popped out previous top. The top of the stack 1 is: " << stack1.top() <<std::endl;
26     std::cout<<"Is the stack 1 empty? " << stack1.empty() <<std::endl;
27     std::cout << "The size of the stack 1 is: " << stack1.size() <<std::endl;
28
29
30     std::stack<int> stack2;
31     std::cout<<"\n===== STACK 2 =====" <<std::endl;
32
33     stack2.emplace(1);
34     std::cout << "Emplaced 1. The top of the stack 2 is: " << stack2.top() <<std::endl;
35     stack2.emplace(2);
36     std::cout << "Emplaced 2. The top of the stack 2 is: " << stack2.top() <<std::endl;
37     stack2.emplace(3);
38     std::cout << "Emplaced 3. The top of the stack 2 is: " << stack2.top() <<std::endl;
39     stack2.emplace(4);
40     std::cout << "Emplaced 4. The top of the stack 2 is: " << stack2.top() <<std::endl;
41
42     std::cout << "\nBefore swap:" <<std::endl;
43     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
44     std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;
45
46     stack1.swap(stack2);
47     std::cout << "\nAfter swap:" <<std::endl;
48     std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
49     std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;
50
51 }
```

In this task, I created and tested two separate stacks to observe how data is stored, managed, and swapped. I started by checking that my first stack was empty, then added four numbers (10, 9, 8, and 7) one by one, confirming that the last number added always stayed on top. Next, I removed the top number (7), leaving 8 as the new top and three items remaining in the stack. I then created a second stack and added four numbers (1, 2, 3, and 4) to it. Finally, I checked the top values of both stacks before swapping their entire contents, successfully demonstrating that the two stacks completely traded all of their items with each other.

Implementation of Emplace and Swap

```
std::stack<int> stack2;
std::cout<<"\n===== STACK 2 =====" <<std::endl;

stack2.emplace(1);
std::cout << "Emplaced 1. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(2);
std::cout << "Emplaced 2. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(3);
std::cout << "Emplaced 3. The top of the stack 2 is: " << stack2.top() <<std::endl;
stack2.emplace(4);
std::cout << "Emplaced 4. The top of the stack 2 is: " << stack2.top() <<std::endl;

std::cout << "\nBefore swap:" <<std::endl;
std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;

stack1.swap(stack2);
std::cout << "\nAfter swap:" <<std::endl;
std::cout << "The top of the stack 1 is: " << stack1.top() <<std::endl;
std::cout << "The top of the stack 2 is: " << stack2.top() <<std::endl;
```

In this part, I focused on populating a second stack and swapping its contents with the first stack. I added the numbers 1, 2, 3, and 4 into the second stack, verifying after each addition that the newest number sat on top. Next, I checked the top value of each stack before swapping them. After executing the swap, I checked the top elements once more to confirm that both stacks had successfully traded their contents.

Output of ILO A

```
----Testing the stack STL----

===== STACK 1 =====
is the stack 1 empty? 1
The top of the stack 1 is: 10
The top of the stack 1 is: 9
The top of the stack 1 is: 8
The top of the stack 1 is: 7
Popped out previous top. The top of the stack 1 is: 8
is the stack 1 empty? 0
The size of the stack 1 is: 3
```

```
===== STACK 2 =====
Emplaced 1. The top of the stack 2 is: 1
Emplaced 2. The top of the stack 2 is: 2
Emplaced 3. The top of the stack 2 is: 3
Emplaced 4. The top of the stack 2 is: 4

Before swap:
The top of the stack 1 is: 8
The top of the stack 2 is: 4

After swap:
The top of the stack 1 is: 4
The top of the stack 2 is: 8
```

This output shows the step-by-step results of my stack operations. First, it confirms that my first stack started off empty (shown as 1), then tracks the top value as I added numbers up to 7. After removing 7, it displays the new top value of 8, confirms the stack is no longer empty (shown as 0), and shows a total count of 3 items remaining. Next, it displays the top value of my second stack updating as I added numbers 1 through 4. Finally, it compares the top values before and after the swap, demonstrating that my first stack's top changed from 8 to 4 while my second stack's top changed from 4 to 8.

ILO B.1: Implement a stack using a Static Array.

Whole code for ILO B.1

```

1 #include <iostream>
2
3 //global declarations
4 #define maxCap 10
5
6
7 int stackArr[maxCap];
8 int top = -1, newData;
9
10 // prototype functions
11 void push();
12 void pop();
13 void Top();
14 bool isEmpty();
15 bool isFull();
16 void display();
17
18 int main(){
19 // main driver
20 int choice;
21 while (true){
22     std::cout<<"===== "<<std::endl;
23     std::cout<<"Stack operations \n";
24     std::cout<<"1. Push\n2. Pop\n3. Top\n4. isEmpty\n5. isFull\n6. Display\n7. Exit program"<<std::endl;
25     std::cout<<"===== "<<std::endl;
26     std::cout<<"\nEnter your choice: ";
27     std::cin >>choice;
28
29     switch(choice){
30         case 1:
31             push();
32             break;
33         case 2:
34             pop();
35             break;
36         case 3:
37             Top();
38             break;
39         case 4:
40             std::cout<<"is stack empty? "<< isEmpty()<<std::endl;
41             break;
42         case 5:
43             std::cout<<"is the stack full? "<< isFull()<<std::endl;
44             break;
45         case 6:
46             display();
47             break;
48         case 7:
49             std::cout<<"Exiting program...";
50             return 0;
51         default:
52             std::cout<<"Invalid choice." <<std::endl;
53             break;
54     }
55 }
56
57 //function definition
58
59
60
61 bool isEmpty(){
62     //how do we verify if the stack is empty
63     if(top == -1) return true;
64     return false;
65 }
66
67 bool isFull(){
68     //how do we verify if stack is full?
69     if (top == maxCap -1) return true;
70     return false;
71 }
72
73 void push(){
74     //error checking ??
75     if(isFull()){
76         std::cout <<"Stack overflow" << std::endl;
77         return;
78     }
79     //pushing to the stack
80     std::cout<<"Enter a new Value: ";
81     std::cin >> newData;
82     std::cout << std::endl;
83
84     // how do we insert the data into the stack?
85     stackArr[++top] = newData;
86 }
87
88 void pop(){
89     //error checking
90     if(isEmpty()){
91         std::cout<<"Stack underflow" <<std::endl;
92     }
93
94     //Display the value that we are going to pop:
95     std::cout<<"Popping: "<<stackArr[top]<<std::endl;
96     //Decrement the top value from the stack
97     top--;
98 }
99
100 void Top(){
101     //error catching:
102     if(isEmpty()){
103         std::cout<<"The stack is empty" <<std::endl;
104         return;
105     }
106     //check the top value
107     std::cout <<"top element: "<<stackArr[top]<<std::endl;
108 }
109
110 void display() {
111     if (isEmpty()) {
112         std::cout << "The stack is empty." << std::endl;
113         return;
114     }
115
116     std::cout << "Stack elements (top to bottom):" << std::endl;
117     for (int i = top; i >= 0; i--) {
118         std::cout << stackArr[i] << std::endl;
119     }
120 }

```

In this task, I built a stack data structure from scratch using a static array with max cap. I implemented all the core stack operations, including adding new elements, removing the top element, viewing the top value, and checking whether the stack is empty or full. To make the program interactive, I created a user menu loop that allows choosing any of these operations in real time, displaying the stack's contents from top to bottom, and cleanly exiting the program when finished.

Output of ILO B.1

```
=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 1
Enter a new Value: 10
Successfully pushed 10

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 2
Popping: 10

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 7
Exiting program...

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 6
Stack elements (top to bottom):
1
2
3
4
5
6
7
8
9
10

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 3
top element: 1

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 4
is stack empty? 0

=====
Stack operations
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display
7. Exit program
=====
Enter your choice: 5
is the stack full? 1
```

This output demonstrates the practical execution of my menu-driven stack program. I tested pushing a value (10) onto the stack and successfully popping it out. I verified the status checks by confirming the stack was not empty (0), checking the current top element (1), and confirming when the stack reached full capacity (1). I also displayed all stored elements vertically from top to bottom (1 through 10), and lastly tested the exit option to cleanly terminate the program.

Implementation of Emplace and Swap

```
void display() {
    if (isEmpty()) {
        std::cout << "The stack is empty." << std::endl;
        return;
    }

    std::cout << "Stack elements (top to bottom):" << std::endl;
    for (int i = top; i >= 0; i--) {
        std::cout << stackArr[i] << std::endl;
    }
}
```

In this part, I created the function responsible for displaying all the elements currently stored in the stack. I first included a check to verify if the stack was empty so it would notify the user if there were no items to show. If the stack contains data, I set it up to traverse backwards from the top index down to the very first item at the bottom, printing each value line by line.

ILO B.2. Stacks using Linked Lists

Whole code for ILO B.2

```
ACTIVITY 4 STACKS > stacks.cpp > ...
1  #include <iostream>
2  #include "stackList.h"
3
4  int main() {
5      std::cout << "---- PUSHING 'c' 'p' 'e' ONTO THE STACK ----\n";
6      //push
7      push('c');
8      push('p');
9      push('e');
10
11     //display the current top
12     Top<char>(); // Top should be 'e'
13     displayAll<char>();
14     std::cout << "\n ----- Popping all the elements ----- \n";
15
16     while (isEmpty<char>() == false) {
17         std::cout << " Popped: " << pop<char>() << std::endl;
18     }
19
20     std::cout << "\n ----- Testing the Underflow ----- \n";
21     pop<char>();
22
23     return 0;
24 }
```

```
1  #ifndef STACKLIST_H
2  #define STACKLIST_H
3
4  #include <iostream>
5
6  template <typename T>
7  struct Node {
8      T data;
9      Node<T>* next;
10 };
11
12 // variable templates so head/tail exist per type (C++14+)
13 template <typename T>
14 Node<T>* head = nullptr;
15
16 template <typename T>
17 Node<T>* tail = nullptr;
18
19 template <typename T>
20 bool isEmpty() {
21     return head<T> == nullptr;
22 }
23
24 template <typename T>
25 void push(T newData) {
26     Node<T>* newNode = new Node<T>;
27     newNode->data = newData;
28     newNode->next = nullptr;
29
30     if (head<T> == nullptr) {
31         head<T> = tail<T> = newNode;
32     } else {
33         newNode->next = head<T>;
34         head<T> = newNode;
35     }
36 }
37
```

```
38 template <typename T>
39 T pop() {
40     if (isEmpty<T>()) {
41         std::cout << "Stack Underflow." << std::endl;
42         return T();
43     }
44
45     Node<T>* temp = head<T>;
46     T tempVal = temp->data;
47     head<T> = head<T>->next;
48     if (head<T> == nullptr) tail<T> = nullptr;
49     delete temp;
50     return tempVal;
51 }
52
53 template <typename T>
54 void Top() {
55     if (isEmpty<T>()) {
56         std::cout << "Stack is Empty." << std::endl;
57         return;
58     }
59     std::cout << "Top of Stack: " << head<T>->data << std::endl;
60 }
61
62 template <typename T>
63 void displayAll() {
64     if (isEmpty<T>()) {
65         std::cout << "The stack is empty." << std::endl;
66         return;
67     }
68
69     std::cout << "Stack elements (top to bottom): " << std::endl;
70     Node<T>* temp = head<T>;
71     while (temp != nullptr) {
72         std::cout << temp->data << std::endl;
73         temp = temp->next;
74     }
75 }
76
77 #endif
```

In this task, I built a stack using a singly linked list, organized into a separate stackList.h header file for reusability. I used C++ templates so push, pop, Top, isEmpty, and displayAll work with any data type, with head<T>/tail<T> as variable templates so each type keeps its own stack. In main.cpp, I tested it by pushing 'c', 'p', 'e', checking the top, displaying all elements, popping everything in a loop, then testing underflow on an already-empty stack.

Output of ILO B.2

```

---- PUSHING 'c' 'p' 'e' ONTO THE STACK ----
Top of Stack: e
Stack elements (top to bottom):
e
p
c

----- Popping all the elements -----
Popped: e
Popped: p
Popped: c

----- Testing the Underflow -----
Stack Underflow.

```

After pushing 'c', 'p', 'e', the top correctly showed 'e', and displayAll() confirmed the order top to bottom as e, p, c. The loop then popped each element in that same order, verifying LIFO behavior. Finally, popping again on the empty stack correctly triggered "Stack Underflow" instead of crashing, confirming the error handling works.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

Whole code for ILO C

```

1  #include <iostream>
2  #include <string>
3  #include "stackList.h"
4
5  using namespace std;
6
7  bool isOpenSymbol(char c) {
8      return (c == '(' || c == '[' || c == '{');
9  }
10
11 bool isCloseSymbol(char c) {
12     return (c == ')' || c == ']' || c == '}');
13 }
14
15 char getExpectedOpener(char closeSym) {
16     switch (closeSym) {
17         case ')': return '(';
18         case ']': return '[';
19         case '}': return '{';
20     }
21     return ' ';
22 }
23
24 void clearStack() {
25     while (!isEmpty<char>()) pop<char>();
26 }
27
28 bool checkBalanced(string input) {
29     clearStack();
30
31     for (char symbol : input) {
32         if (isOpenSymbol(symbol)) {
33             push<char>(symbol);
34             continue;
35         }
36
37         if (isCloseSymbol(symbol)) {
38             if (isEmpty<char>()) return false;
39
40             char lastOpened = pop<char>();
41             if (lastOpened != getExpectedOpener(symbol)) return false;
42         }
43     }
44
45     return isEmpty<char>();
46 }
47

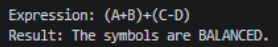
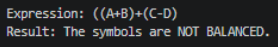
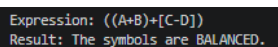
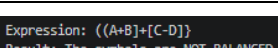
```

```

48 int main() {
49     string testCases[] = {
50         "(A+B)+(C-D)",
51         "((A+B)+(C-D))",
52         "((A+B)+(C-D))",
53         "((A+B)+(C-D))",
54     };
55
56     int totalCases = 4;
57
58     for (int i = 0; i < totalCases; i++) {
59         cout << "Expression: " << testCases[i] << endl;
60
61         if (checkBalanced(testCases[i]))
62             cout << "Result: The symbols are BALANCED." << endl;
63         else
64             cout << "Result: The symbols are NOT BALANCED." << endl;
65     }
66
67     return 0;
68 }

```

In this task, I created a program to check if an expression has balanced symbols using the linked list stack from `stackList.h`. I made functions to check if a character is an opening or closing symbol, and a `getExpectedOpener()` function that returns the matching opening symbol for a given closing one. The `checkBalanced()` function clears the stack first, then pushes opening symbols and pops when it hits a closing symbol, checking if they match. If the stack isn't empty at the end, the expression is not balanced. In `main()`, I stored the four test expressions in an array and looped through them to test all cases at once instead of asking for user input.

Self-Checking:			
For the following cases, complete the table using the code you created.			
Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
<code>(A+B)+(C-D)</code>	Y		Every (has a matching) in order. Stack ends empty, so it's balanced.
<code>((A+B)+(C-D)</code>	N		Three (but only two). One (is left on the stack at the end, so it's unbalanced.
<code>((A+B)+[C-D])</code>	Y		All symbols are closed in the correct nested order. Stack ends empty, so it's balanced.
<code>((A+B)+[C-D])}</code>	N		} tries to close a (on the stack instead of [. Mismatch caught immediately, so it's unbalanced.

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?

The array stack has a fixed size, so we need to check if it's full before pushing. The linked list version has no size limit but uses more memory per element and we have to manage deleting nodes ourselves. The STL stack was the easiest since `push`, `pop`, and `top` are already built in. All three gave the same balanced/not balanced results, since what matters is the push/pop order, not how the stack is stored.

Summary of Lessons Learned

Working through this activity gave me a much clearer picture of how stacks actually work under the hood. Beyond just knowing the basic LIFO (last-in, first-out) rule where whatever goes in last comes out first, I got to build stacks in three distinct ways: using a fixed array, a dynamic linked list, and C++'s built-in STL `std::stack`. It was really cool seeing how core operations like `push`, `pop`, `top`, and `isEmpty` do the same job across all three versions, even though their inner mechanics are completely different.

Building the array version made me appreciate guard checks for stack overflow and underflow, given the fixed size limits. Switching to the linked list forced me to pay close attention to dynamic memory management, specifically cleaning up deleted nodes to prevent memory leaks. I also used C++ templates there, which let the stack handle any data type rather than being locked to just one. Finally, plugging these custom stacks into a balanced symbol checker gave all that theory a practical purpose, showing how simple push and pop actions can solve real string validation problems.

Analysis of the Procedure

The lab steps felt very easy to do because each step built smoothly on the last. Starting with the C++ STL stack was a great warm-up since it let me focus purely on high-level stack behavior. Moving straight into the manual array implementation made it easy to see how to build those same operations from scratch. By the time I moved on to the linked list version, the core stack operations were already second nature, so I could focus on wrapping my head around dynamic pointers and generic template syntax without getting flooded down by the basic logic.

Following this sequence also made testing and debugging a lot smoother. Because I had reliable reference behavior from the earlier steps, I could easily check if my custom implementations were outputting the right results at every stage. Also the guidance of professor made this part easy to do.

Analysis of the Supplementary Activity

The supplementary exercise on building a balanced symbol checker was easily the highlight of the lab. It bridged the gap between raw data structures and a real application. It forced me to think through edge cases rather than just happy paths, such as how the program should react when it encounters a closing symbol while the stack is empty, or when a popped opening bracket does not pair up with the current closing character.

Running the checker across all four test expressions gave me immediate visual feedback on whether my conditional logic held up. Reusing my custom templated linked list stack for this task was also satisfying because it showed the value of writing modular, reusable code from the start.

Concluding Statement / Feedback

Overall, the activity went smoothly. All three implementations (array, linked list, and STL) behaved consistently and passed the symbol checker without hitching. The main takeaway for me was that regardless of how a stack is structured under the hood, its interface and LIFO guarantee remain rock solid.

If there is one area I know I need to polish, it is writing templated code in C++. Navigating template syntax and variable templates like `head<T>` and `tail<T>` took extra time and effort compared to the straightforward array implementation. Moving forward, I want to spend more time practicing generic programming in C++ so I can write reusable, type-agnostic structures more fluidly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.